

Smart Parking System



Figure 1: Team 313

Rizvi Ahmed (ID: 0112310390)

Mohammud Ali (ID: 0112310087)

A S M Fakhruddin (ID: 0112310375)

Electronics Lab

Section: (D)

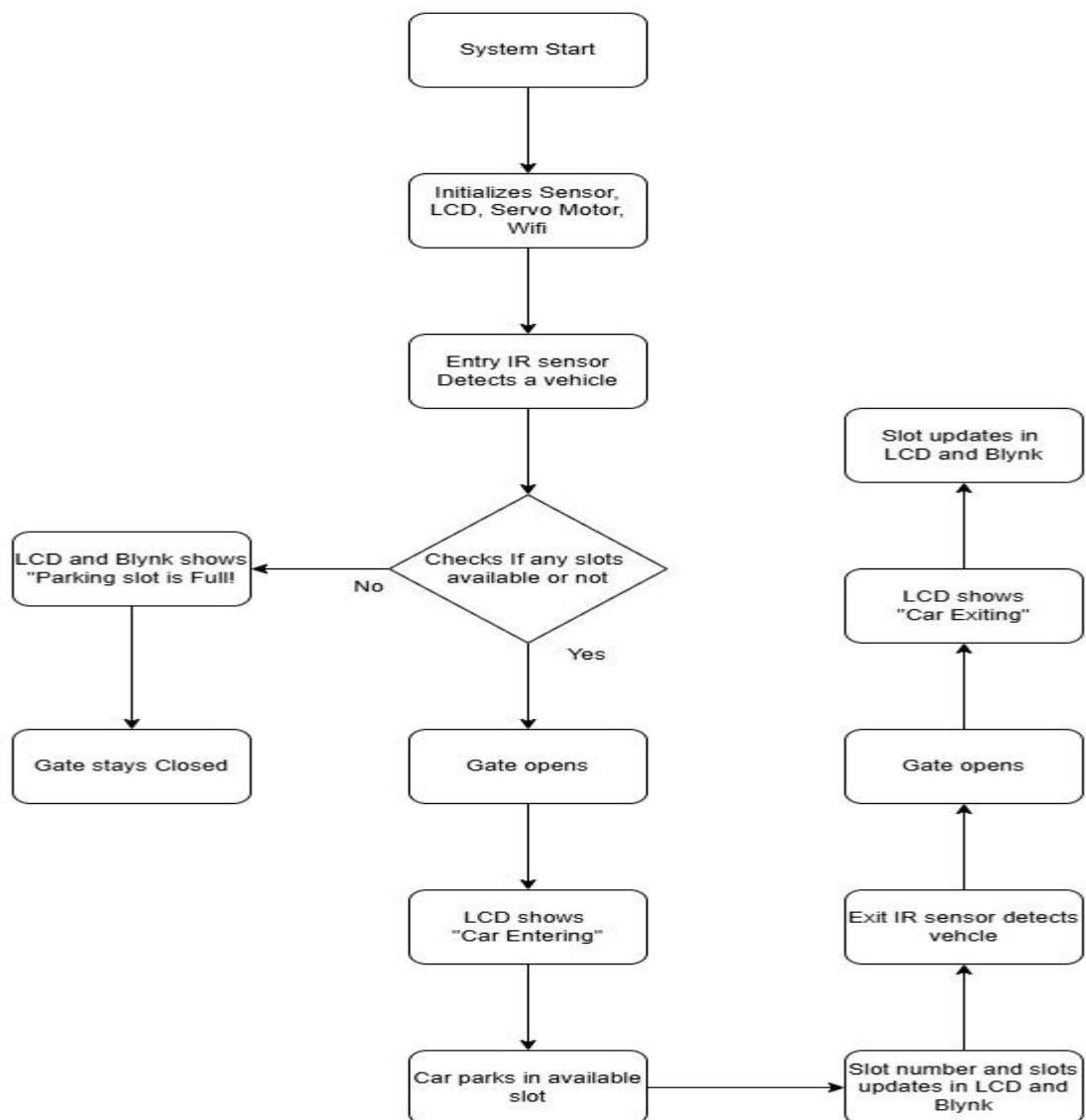
Computer Science and Engineering

United International University

Abstract— SmartPark is an IoT-based intelligent parking management system designed to automate the detection, monitoring, and management of parking spaces in real time. The system integrates ultrasonic sensors, an ESP32 microcontroller, servo motors, and the Blynk IoT platform to provide an efficient and user-friendly parking experience. By automating gate control and slot detection, the system minimizes manual supervision, reduces congestion, and improves time management for vehicle parking. Real-time data is sent to the user's smartphone via the Blynk app, allowing remote monitoring of available parking slots. This project demonstrates how embedded systems and IoT can enhance urban mobility and optimize space utilization.

I. PROJECT OVERVIEW

The SmartPark system is designed to automate the parking process using a network of sensors and an IoT-based monitoring platform. When a vehicle approaches the entry gate, an IR sensor detects its presence and activates the servo motor to open the gate if space is available. Each parking slot is equipped with an ultrasonic sensor that determines whether the slot is occupied or vacant. The ESP32 microcontroller collects this data and updates it in real time on the Blynk app dashboard. If all slots are full, the display screen shows “Parking Full” and the entry gate remains closed. Similarly, when a vehicle leaves, the exit sensor detects the motion, updates the database, and reopens availability for the next user.



II. COMPONENT LIST

A. ESP32 Microcontroller

The ESP32 is a powerful Wi-Fi-enabled microcontroller used as the brain of the system. It connects sensors and actuators and communicates with the Blynk cloud server for real-time data transmission.

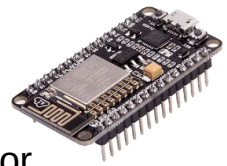


Figure 2 : Esp32 Microcontroller

B. Ultrasonic Sensors

Ultrasonic sensors detect the presence of vehicles in parking slots by measuring distance using sound waves. When the measured distance falls below a threshold, the system identifies the slot as occupied.



Figure 3: Ultrasonic Sensor

C. IR Sensors

Infrared sensors are used at the entry and exit gates to detect vehicle movement. These sensors trigger the servo motor for gate operations.



Figure 4: IR Sensor

D. Servo Motor

The servo motor is responsible for automatic gate control. It opens the gate when a slot is available and closes it once the car enters or exits.



Figure 5: Servo Motor

E. LCD Display

The LCD module displays real-time parking status, such as available slots and system messages like "Parking Full."



Figure 6: LCD Display

F. Blynk IoT Platform

The Blynk app provides a user interface to view parking slot status remotely. It connects via Wi-Fi to the ESP32, updating the user with live data.

III. FEATURES

The SmartPark system integrates multiple smart features to improve parking efficiency and automation:

1. **Automatic Gate Control:** The entry gate opens and closes automatically based on slot availability.
2. **Smart Slot Detection:** Each parking slot's status is detected and updated in real time using ultrasonic sensors.
3. **Display and Alert System:** LCD display and Blynk notifications provide real-time updates to users.
4. **IoT Connectivity:** Users can monitor parking availability remotely through the Blynk app.
5. **Energy Efficiency:** The system only activates components when needed, reducing unnecessary power consumption.

IV Code

ESP32 Wi-Fi and Blynk Setup Code

```
1  #include <Wire.h>
2  #include <LiquidCrystal_I2C.h>
3
4
5  LiquidCrystal_I2C lcd(0x27, 20, 4);
6
7  #define BLYNK_TEMPLATE_ID "TMPL6j_264BEV"
8  #define BLYNK_TEMPLATE_NAME "ParkSmart"
9  #define BLYNK_AUTH_TOKEN   "NfoqXgB5jxKNvQE1DYAH5-8jd6vwA1sd"
10
11 #define BLYNK_PRINT Serial
12 #include <WiFi.h>
13 #include <BlynkSimpleEsp32.h>
14 #include <ESP32Servo.h>
15
16 char ssid[] = "ParkEasy";
17 char pass[] = "11111111";
18
```

Figure 7: Esp wifi and Blynk setup code

```

104
105 void setup() {
106     Serial.begin(115200);
107
108
109     Wire.begin(21, 22);
110     lcd.init();
111     lcd.backlight();
112     lcd.setCursor(0,0);
113     lcd.print("System Starting...");
114
115     pinMode(IR_ENTER, INPUT);
116     pinMode(IR_BACK, INPUT);
117     pinMode(TRIG1, OUTPUT); pinMode(ECHO1, INPUT);
118     pinMode(TRIG2, OUTPUT); pinMode(ECHO2, INPUT);
119     pinMode(TRIG3, OUTPUT); pinMode(ECHO3, INPUT);
120     pinMode(TRIG4, OUTPUT); pinMode(ECHO4, INPUT);
121     pinMode(TRIG5, OUTPUT); pinMode(ECHO5, INPUT);
122
123     gateServo.attach(SERVO_PIN);
124     gateServo.write(150);
125
126     Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);
127
128     timer.setInterval(2000L, sendStatusToBlynk);
129
130     debugPrint("System Initialized");
131 }
132

```

Figure 8: Setup Function — Hardware and Sensor Initialization

```

133 void loop() {
134     Blynk.run();
135     timer.run();
136
137     updateSlots();
138
139     if (digitalRead(IR_ENTER) == LOW && flag1==0) {
140         if (slot>0) {
141             flag1=1;
142             if (flag2==0) {
143                 gateServo.write(0);
144                 debugPrint("Car Entering...");
145                 delay(4000);
146                 gateServo.write(150);
147                 debugPrint("Gate Closed");
148             }
149             else {
150                 debugPrint("Parking space Full!");
151             }
152         }
153
154         if (digitalRead(IR_BACK) == LOW && flag2==0) {
155             flag2=1;
156             gateServo.write(0);
157             debugPrint("Car Exiting...");
158             delay(4000);
159             gateServo.write(150);
160             debugPrint("Gate Closed");
161         }
162
163         if (digitalRead(IR_ENTER)==HIGH) flag1=0;
164         if (digitalRead(IR_BACK)==HIGH) flag2=0;
165     }
166

```

Figure 9: Main Loop — Real-time Operation

```
21 Servo gateServo;
22 #define SERVO_PIN 14
23
24 #define IR_ENTER 16
25 #define IR_BACK 13
26
27 #define TRIG1 4
28 #define ECHO1 34
29 #define TRIG2 5
30 #define ECHO2 35
31 #define TRIG3 18
32 #define ECHO3 32
33 #define TRIG4 19
34 #define ECHO4 33
35 #define TRIG5 23
36 #define ECHO5 25
37
```

Figure 10: System Connection pins

IV. CONCLUSION

The SmartPark project demonstrates how IoT technology can be applied to everyday challenges such as urban parking management. Through real-time monitoring, automated gate control, and mobile connectivity, the system minimizes manual effort and improves user convenience. This solution offers a scalable and affordable approach for implementation in malls, offices, and public parking areas. With further enhancements such as camera-based detection and payment automation, SmartPark can evolve into a fully integrated smart parking infrastructure.